Table 3: Memory and time complexity of Transformer variants. We write d_{model} and d_{ff} for model depth and assume $d_{ff} \geq d_{model}$; b stands for batch size, l for length, n_l for the number of layers. We assume $n_c = l/32$ so $4l/n_c = 128$ and we write $c = 128^2$.

Model Type	Memory Complexity	Time Complexity
Transformer	$\max(bld_{ff}, bn_h l^2)n_l$	$(bld_{ff} + bn_h l^2)n_l$
Reversible Transformer	$\max(bld_{ff}, bn_h l^2)$	$(bn_h ld_{ff} + bn_h l^2)n_l$
Chunked Reversible Transformer	$\max(bld_{model}, bn_h l^2)$	$(bn_h ld_{ff} + bn_h l^2)n_l$
LSH Transformer	$\max(bld_{ff}, bn_h ln_r c)n_l$	$(bld_{ff} + bn_h n_r lc)n_l$
Reformer	$\max(bld_{model}, bn_h ln_r c)$	$(bld_{ff} + bn_h n_r lc)n_l$

4 Related Work

The Transformer model introduced in (Vaswani et al., 2017) has been used widely in natural language tasks and further extended to model diverse data such as music scores (Huang et al., 2018), and images (Parmar et al., 2018; Ramachandran et al., 2019). Most notably, this model class has been applied successfully in the self-supervised training of extremely large language models (Devlin et al., 2018; Radford et al., 2019).

Given the enormous computational requirements of state of the art sequence models, there has been increasing interest in finding methods to reduce the memory footprint and computational requirements of Transformer models. In addition to standard methods such as precision reduction and gradient checkpointing (Sohoni et al., 2019), more efficient versions of the Transformer model's self-attention mechanism (Sukhbaatar et al., 2019a;b) have also recently been explored.

In particular, leveraging sparsity in the attention layers has proved fruitful. OpenAI introduced the sparse Transformer (Child et al., 2019) which exploits a factorized sparse representation of attention. Using product-key attention to increase the key space has also been used to reduce memory requirements in the feed-forward layers with no loss in performance (Lample et al., 2019).

Locality-sensitive hashing (LSH) has, to our knowledge, not been directly applied to Transformer attention layers before. But previous work using external memory with neural networks has dealt with memories of large sizes. The original implementation of memory networks (Weston et al., 2014) and later work on scaling it (Bordes et al., 2015; Chandar et al., 2016) used memory with size in the millions. The cost of doing so is that the memory must be fixed prior to training. Moreover, since during the beginning of training the model is unlikely to query the memory correctly, strong supervision is used to encourage the model to query memory locations that are useful. These hints are either given as additional supervising information by the task or determined heuristically as in Hill et al. (2015). The requirement that the memory be fixed before has been removed in Santoro et al. (2016) at the cost of memory size and later alleviated by Rae et al. (2016). The last paper considered memory lookups with approximate nearest neighbors including both LSH and random kd-trees, but only for lookups in external memory.

5 Experiments

In this section we present experimental results demonstrating the techniques described above. We analyze the techniques one-by-one to make clear which combinations have impact on performance. We start by showing that reversible layers and shared query-key spaces do not impact performance, then proceed to analyze hashing attention and finally the full Reformer model.

We ran our experiments on the imagenet64 and enwik8-64K tasks, where the latter is a variant of enwik8 that is chunked into subsequences of $2^{16} = 64K$ tokens. We use 3-layer models for our ablations so as to make it tractable to compare with the regular Transformer, which has high memory usage and performs full $O(l^2)$ attention. All experiments have $d_{model} = 1024$, $d_{ff} = 4096$, $n_{heads} = 8$, and a total batch size of 8 sequences. We used the Adafactor optimizer (Shazeer & Stern, 2018) for training these models. We also evaluate on the WMT 2014 English-to-German translation task, following the hyperparameters of Vaswani et al. (2017). Training for all experiments

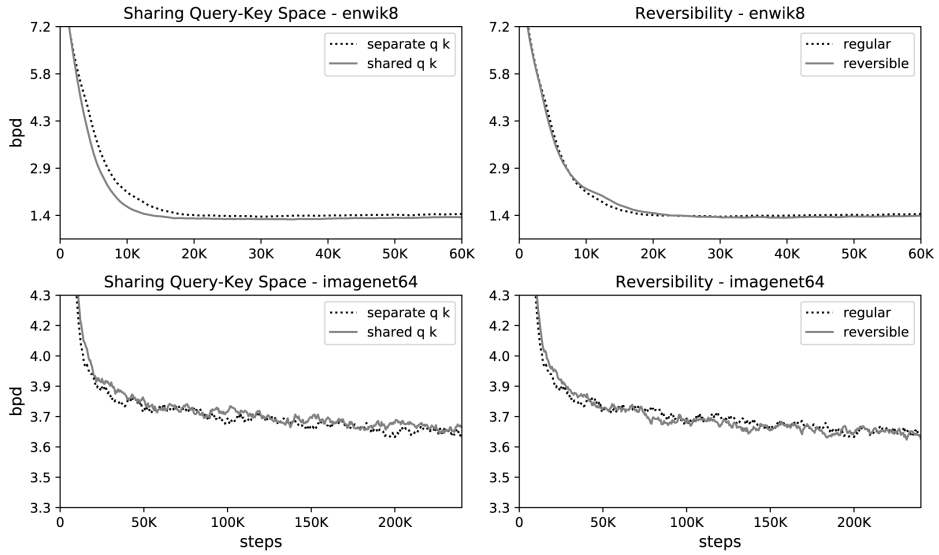

Figure 3: Effect of shared query-key space (left) and reversibility (right) on performance on enwik8 and imagenet64 training. The curves show bits per dim on held-out data.

Table 4: BLEU scores on newstest2014 for WMT English-German (EnDe). We additionally report detokenized BLEU scores as computed by sacreBLEU (Post, 2018).

		sacreBLEU	
Model	BLEU	*Uncased*[3]	*Cased*[4]
Vaswani et al. (2017), base model	27.3		
Vaswani et al. (2017), big	28.4		
Ott et al. (2018), big	29.3		
Reversible Transformer (base, 100K steps)	27.6	*27.4*	*26.9*
Reversible Transformer (base, 500K steps, no weight sharing)	28.0	*27.9*	*27.4*
Reversible Transformer (big, 300K steps, no weight sharing)	29.1	*28.9*	*28.4*

was parallelized across 8 devices (8 GPUs or 8 TPU v3 cores). Code for training our models is made publicly available.[2]

Effect of sharing QK. We first consider the effect of shared-QK attention on a regular Transformer model. Shared-QK attention sets $k_j = \frac{q_j}{\|q_j\|}$ and prevents tokens from attending to themselves (except when no other context is available). In the left part of Figure 3, we plot perplexity curves for both regular and shared-QK attention. A shared query-key space does not perform worse than regular attention; in fact, for enwik8 it appears to train slightly faster. In other words, we are not sacrificing accuracy by switching to shared-QK attention.

Effect of reversible layers. In the two plots on the right in Figure 3, we compare a regular Transformer per Vaswani et al. (2017) with the reversible one describe in Section 3. The two models have identical parameter counts, and the learning curves likewise appear to be nearly the same. These results show that the memory savings in the reversible Transformer do not come at the expense of accuracy.

Reversible layers in machine translation. We also evaluate reversible layers in the context of an encoder-decoder Transformer model for machine translation from English to German. We start by making both the encoder and the decoder fully reversible in the Transformer-base architecture, and

[2] `https://github.com/google/trax/tree/master/trax/models/reformer`

[3] `BLEU+case.lc+lang.en-de+numrefs.1+smooth.exp+test.wmt14/full+tok.intl+version.1.4.3`

[4] `BLEU+case.mixed+lang.en-de+numrefs.1+smooth.exp+test.wmt14/full+tok.intl+version.1.4.3`

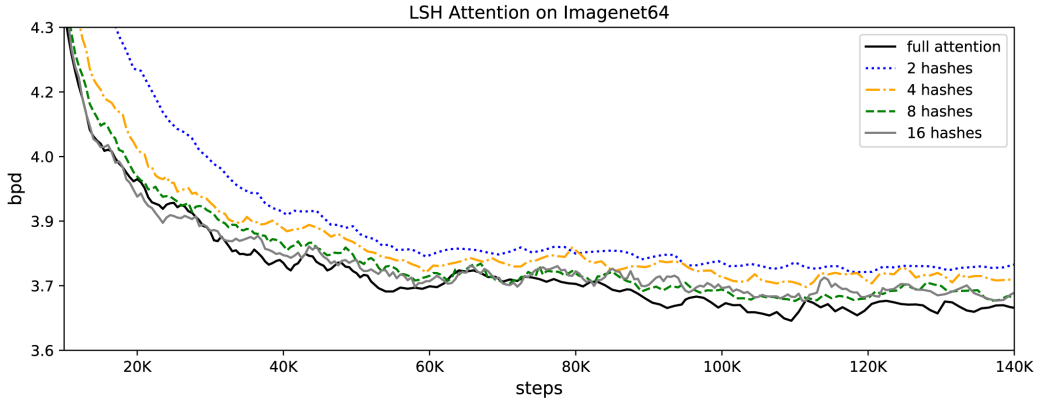

Figure 4: LSH attention performance as a function of hashing rounds on imagenet64.

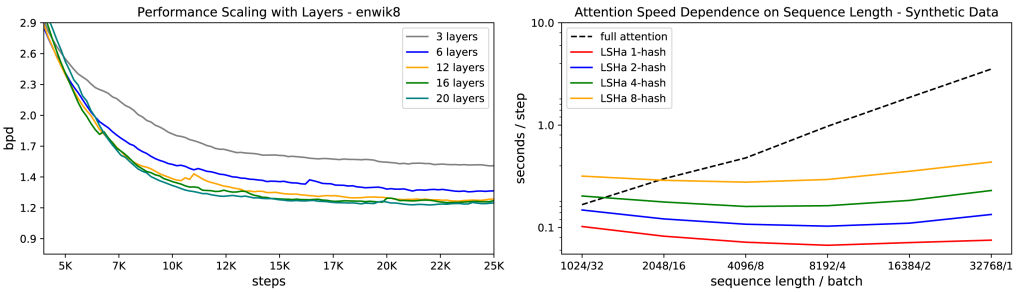

Figure 5: Left: LSH attention performance as a function of number of layers on enwik8. Right: Speed of attention evaluation as a function of input length for full- and LSH- attention.

see that the resulting model performs comparably to Vaswani et al. (2017) when trained for 100K steps. We also evaluate training for a greater number of steps and with a larger model. Reformer models are very memory-efficient, so for the latter two experiments we do not need to save memory by sharing embedding and output projection weight matrices throughout the model. Results are shown in Table 4. We do not apply LSH attention in this setting because examples are single sentences, and sentences tend to be relatively short. Our typical LSH attention configuration uses chunks of 128 tokens after hashing and sorting, whereas the examples in the WMT14 test set are all shorter than 128 tokens.

LSH attention in Transformer. LSH attention is an approximation for full attention that, as evidenced in Figure 4, becomes more accurate as the number of hashes increases. At $n_{rounds} = 8$, it already almost matches full attention. The computational cost of a model grows with the number of hashes, so this hyperparameter can be adjusted depending on the available compute budget. Additionally, as in Table 2, the number of hashes can be increased at evaluation time to produce more accurate results. On the right half of Figure 5, we plot the speed of different attention types vs. the sequence length, while holding the total number of tokens fixed. We see that while regular attention becomes slower at longer sequence length, LSH attention speed remains flat.

Large Reformer models. To verify that the Reformer can indeed fit large models on a single core and train fast on long sequences, we train up to 20-layer big Reformers on enwik8 and imagenet64. As can be seen in Figure 5, these models fit into memory and train. We were not able to train Transformer baselines in this case as they are too slow and memory-hungry, but we see clear improvement with the number of layers. A 12-layer model on enwik8 trained for 20K steps with a dropout rate of 0.1 achieves 1.19 bits/dim on the test set. We also trained a 12-layer Reformer model for longer with further tuning and improvements and we reached 1.05 bits/dim on the enwiki8 test set.